

1. What is Continuous Integration (CI)

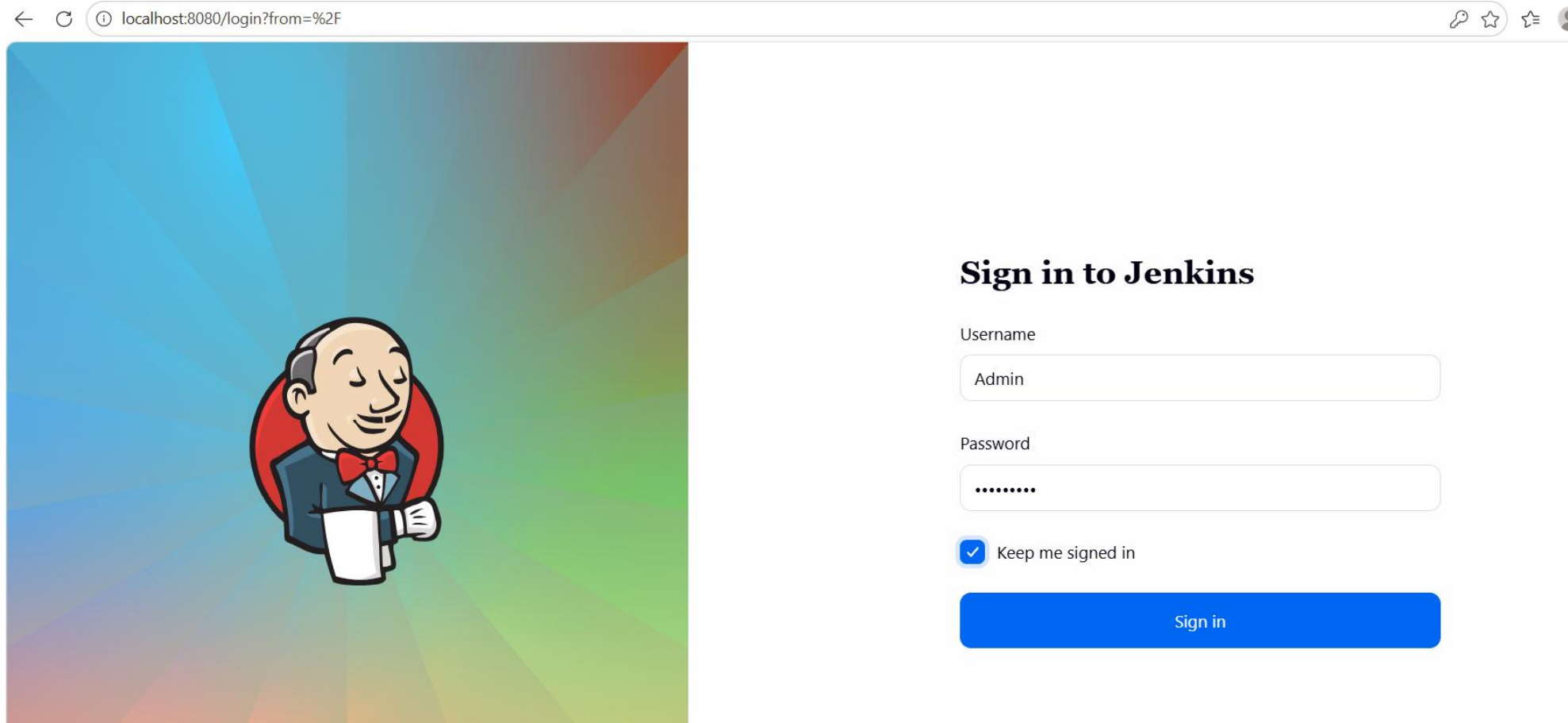
Definition:

Continuous Integration is a process where developers **frequently integrate code**, and Jenkins automatically:

- Builds the project
- Runs tests
- Reports results

2. Open Jenkins Dashboard

http://localhost:8080



A screenshot of a web browser displaying the Jenkins login page. The browser's address bar shows the URL 'localhost:8080/login?from=%2F'. The page features a large, colorful abstract background on the left with a cartoon illustration of a man in a tuxedo and bow tie. On the right, there is a 'Sign in to Jenkins' section with a 'Username' field containing 'Admin', a 'Password' field with masked characters, a checked 'Keep me signed in' checkbox, and a blue 'Sign in' button.

← ↻ ⓘ localhost:8080/login?from=%2F 🔑 ☆ ≡

Sign in to Jenkins

Username

Admin

Password

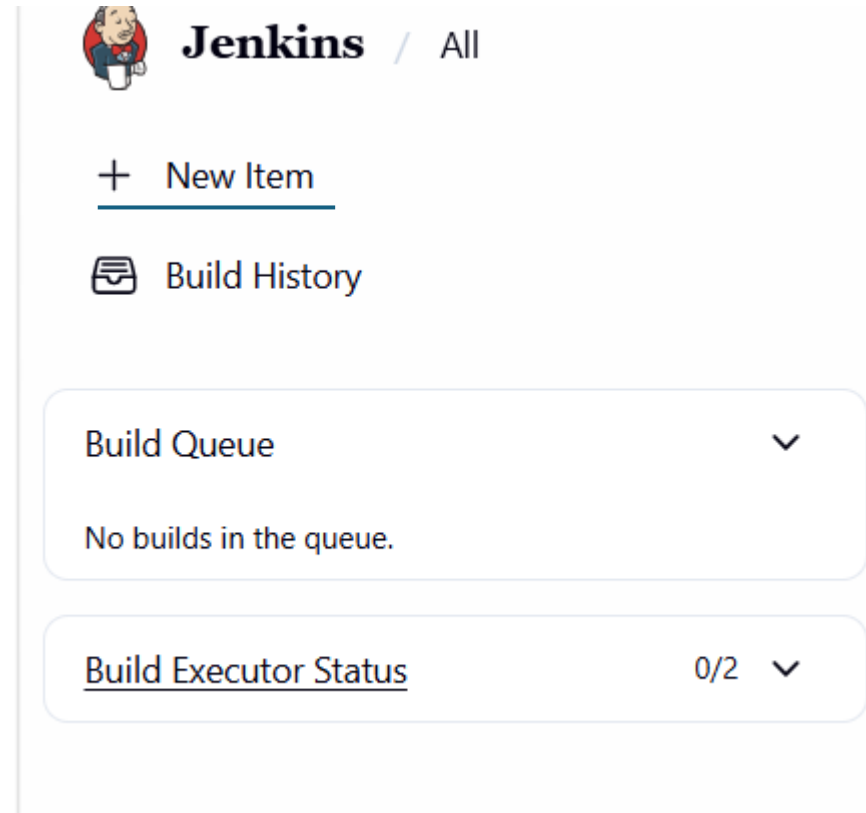
.....

☒ Keep me signed in

Sign in

3. Create New CI Project

- Click **New Item**



Enter an item name:

- Select:
Freestyle Project

- Click **OK**

 **Jenkins** / All  / New Item

New Item

Enter an item name

Select an item type



Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

OK

4. Configure Source Code (Git Integration)

Scroll to:

Source Code Management

Select **Git**

Enter repository URL:



Jenkins / Project_6 ▾ / Configure

Configure



General



Source Code Management



Triggers



Environment



Build Steps



Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.



None



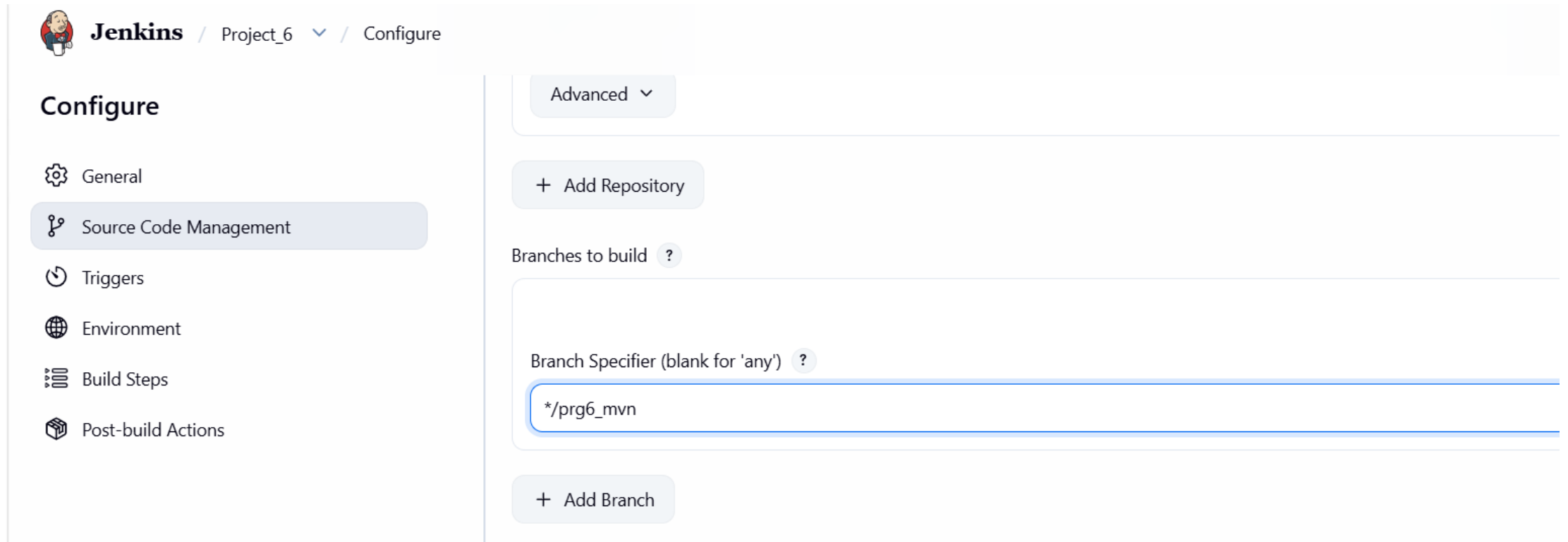
Git ?

Repositories ?

Repository URL ?

`https://github.com/your-repo/sample-project.git`

- **Specify the Git branch name**



The screenshot displays the Jenkins web interface for configuring a project. The breadcrumb navigation at the top shows 'Jenkins / Project_6 / Configure'. On the left sidebar, the 'Configure' section is active, with sub-options: General, Source Code Management (highlighted), Triggers, Environment, Build Steps, and Post-build Actions. The main content area is titled 'Advanced' and contains a '+ Add Repository' button. Below this, the 'Branches to build' section is expanded, showing a 'Branch Specifier (blank for 'any')' field with the value '*/prg6_mvn'. A '+ Add Branch' button is located at the bottom of this section.

Jenkins / Project_6 / Configure

Configure

- General
- Source Code Management**
- Triggers
- Environment
- Build Steps
- Post-build Actions

Advanced ▾

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/prg6_mvn

+ Add Branch

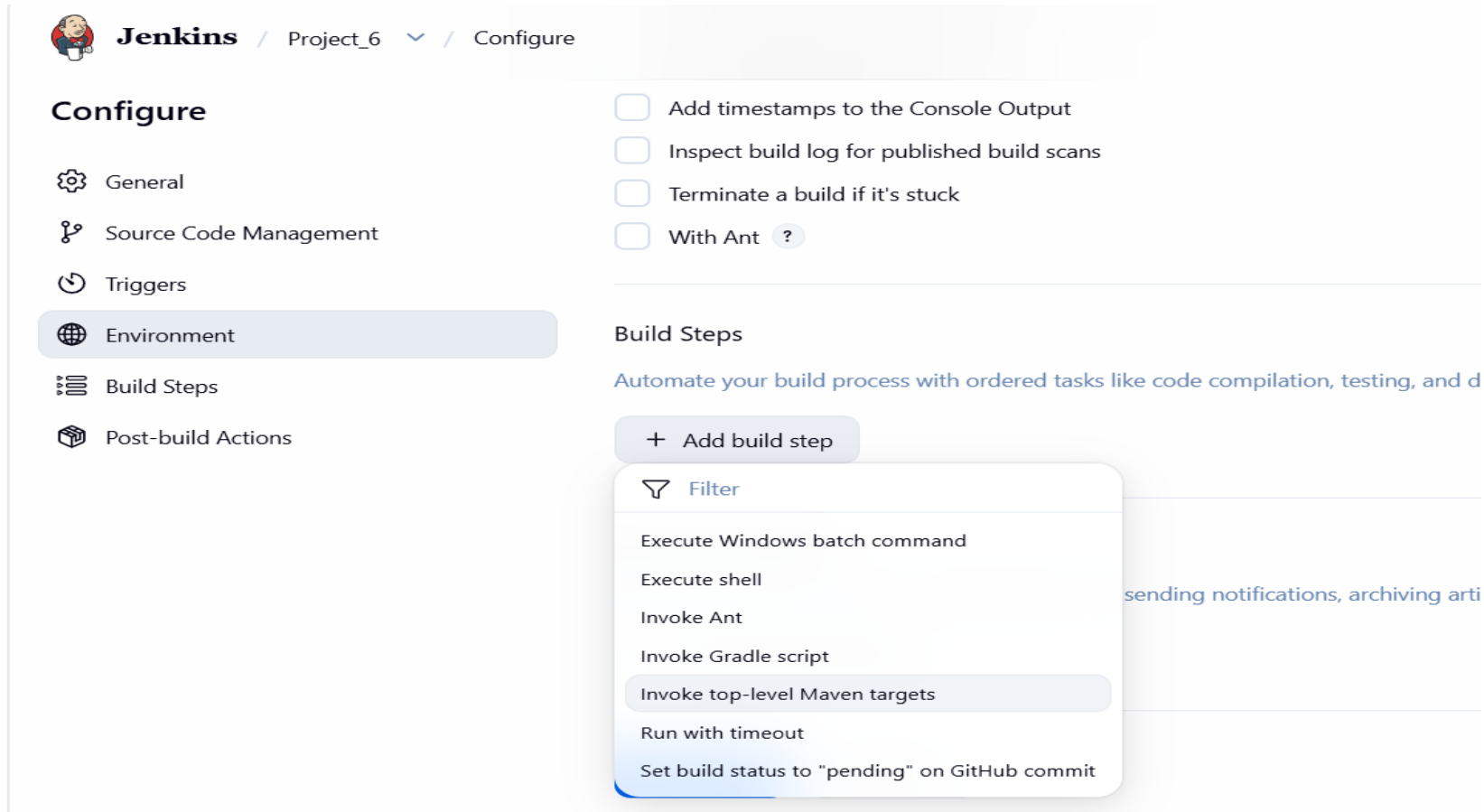
5. Add Build Tool (Maven Integration)

Scroll to:

Build Section

Click:

👉 **Add Build Step → Invoke top-level Maven targets**



The screenshot shows the Jenkins configuration interface for a project named "Project_6". The breadcrumb navigation at the top reads "Jenkins / Project_6 / Configure". On the left sidebar, under the "Configure" heading, the "Environment" tab is selected, with other options like "General", "Source Code Management", "Triggers", "Build Steps", and "Post-build Actions" listed below it. The main content area is divided into two sections: "Configure" and "Build Steps". The "Configure" section contains four unchecked checkboxes: "Add timestamps to the Console Output", "Inspect build log for published build scans", "Terminate a build if it's stuck", and "With Ant" (which has a question mark icon). The "Build Steps" section has a descriptive text: "Automate your build process with ordered tasks like code compilation, testing, and d". Below this text is a button labeled "+ Add build step". A dropdown menu is open from this button, displaying a list of build steps. The list includes "Execute Windows batch command", "Execute shell", "Invoke Ant", "Invoke Gradle script", "Invoke top-level Maven targets" (which is highlighted with a light blue background), "Run with timeout", and "Set build status to 'pending' on GitHub commit".

Goals: Clean install

This command: Cleans old build, Compiles code, Runs tests
Creates build file

Click on: Advanced

Enter pom.xml to read the pom file

The screenshot shows the Jenkins web interface for configuring 'Project_6'. The breadcrumb navigation at the top reads 'Jenkins / Project_6 / Configure'. On the left, a sidebar lists configuration categories: General, Source Code Management, Triggers, Environment, Build Steps (which is highlighted), and Post-build Actions. The main content area is titled 'Build Steps' and includes a description: 'Automate your build process with ordered tasks like code compilation, testing, and deployment.' Below this, there is a section for 'Goals' with a single entry 'Clean install'. A tab labeled 'Advanced' is selected, and next to it is an 'Edited' status with a pencil icon. Under the 'Advanced' tab, there is a 'POM' section with a text input field containing 'pom.xml'. Below the POM section is a 'Properties' section, which is currently empty.

Jenkins / Project_6 / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps**
- Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Invoke top-level Maven targets ?

Goals

Clean install

Advanced ^ Edited

POM ?

pom.xml

Properties ?

(Optional) Gradle Integration

Instead of Maven:

Choose:

 **Invoke Gradle Script**

Save the Project

Click:

 **Save**

6. Run CI Pipeline (Build Now)

Click:

Build Now

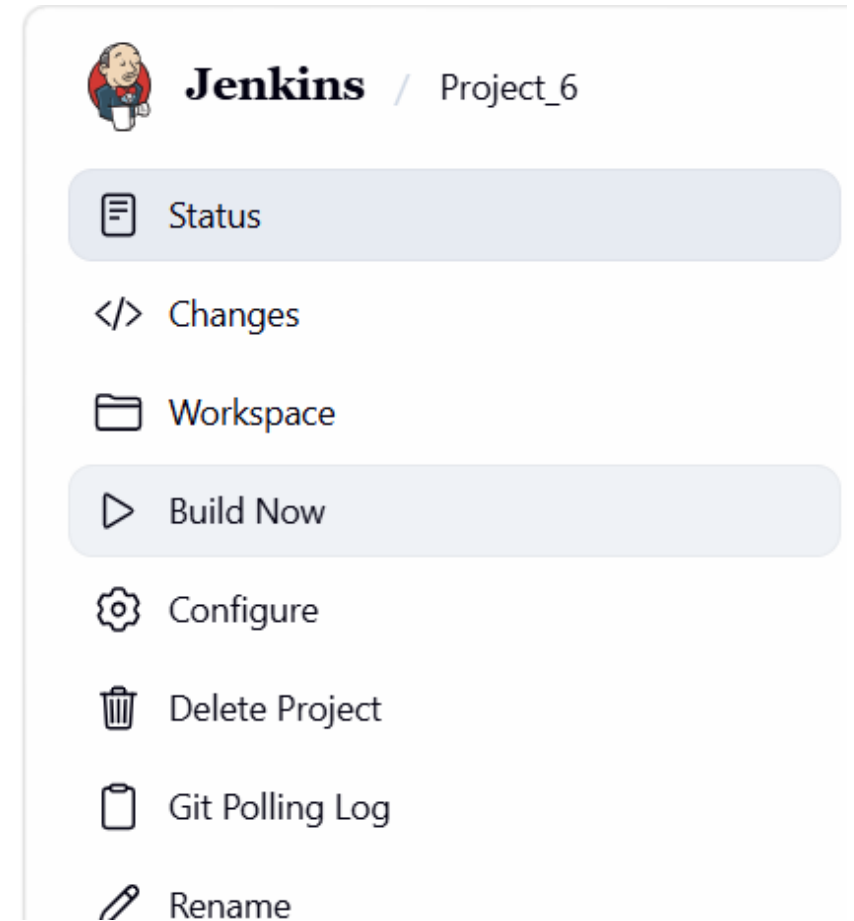
7. View Console Output (Most Important)

Click:

Build #1

Click:

Console Output



Expected Output (Maven Example)

Started by user admin
Cloning the repository
Building project using Maven
Running tests...
BUILD SUCCESS
Finished: **SUCCESS**

CI Pipeline Flow

- Code Push → Jenkins Pull → Build → Test → Result